

NLP TEXT CLASSIFICATION & AUTOMATION

Professional Technical Report

An end-to-end machine-learning workflow for transforming unstructured text into structured categories and reusable automated predictions.

18,846	20	76.22%
Initial documents	Topic classes	Linear SVM accuracy

Core technologies: Python · NLTK · scikit-learn · TF-IDF · Linear SVM · Joblib

Source: User-supplied NLP Text Classification Pipeline notebook/PDF

01 Executive Summary

The supplied notebook implements a complete Natural Language Processing (NLP) text-classification pipeline using the 20 Newsgroups benchmark. The workflow progresses from raw documents through cleaning and linguistic normalization, numerical feature representation with TF-IDF, supervised model training, evaluation, persistence, and inference on previously unseen text.

The source run starts with 18,846 documents distributed across 20 categories. After removing empty documents and cleaning the text, 18,290 documents remain. An 80:20 stratified split produces 14,632 training documents and 3,658 test documents. Among the three evaluated baseline models, Linear SVM reports the highest accuracy at 76.22%.

Automation perspective: the demonstrated inference function provides the decision layer for an automated text-routing workflow: incoming text is cleaned, converted using the saved TF-IDF vectorizer, classified by the saved model, and then made available for a downstream action.

Project Snapshot

Dataset	20 Newsgroups
Initial records	18,846
Classes	20
After empty-document removal	18,331
After text cleaning	18,290
Train / Test	14,632 / 3,658
Feature representation	TF-IDF, max 20,000 features, 1–2 grams
Best reported model	Linear SVM
Best reported accuracy	76.22%

02 Solution Architecture

The implementation is modular: each stage has a defined responsibility, making the workflow easier to test, reuse, and adapt to domain-specific automation.

01	RAW TEXT	Incoming document / message
02	PREPROCESSING	Normalize and remove linguistic noise
03	TF-IDF	Convert text into weighted numerical features
04	CLASSIFIER	Predict the most appropriate category
05	PERSISTENCE	Store model + vectorizer for reuse
06	AUTOMATION	Route, tag, trigger or prioritize

Pipeline principle: Text → Clean → Vectorize → Classify → Persist → Automate

03 Data Preparation & Quality

The source removes headers, footers and quotes while loading the benchmark. Exploratory analysis includes category distribution and basic character/word-length statistics. The quality check reports no missing text values, while 515 documents are empty strings and are removed before modeling.

```
# Remove empty documents
print("Missing text values:", df["text"].isna().sum())
print("Empty-string documents:",
      (df["text"].str.strip() == "").sum())

df = df[df["text"].str.strip() != ""].reset_index(drop=True)
print(f"Documents remaining: {len(df):,}")
```

04 NLP Preprocessing

The cleaning function standardizes incoming text before feature extraction. The supplied implementation performs lowercasing, URL removal, email removal, punctuation removal, number removal, whitespace normalization, stopword filtering and WordNet lemmatization.

Preprocessing code

```
stop_words = set(stopwords.words("english"))
lemmatizer = WordNetLemmatizer()

def clean_text(text):
    text = text.lower()
    text = re.sub(r"http\S+|www\S+", " ", text)
    text = re.sub(r"\S+@\S+", " ", text)
    text = re.sub(
        f"[{re.escape(string.punctuation)}]", " ", text
    )
    text = re.sub(r"\d+", " ", text)
    text = re.sub(r"\s+", " ", text).strip()

    tokens = text.split()
    tokens = [
        lemmatizer.lemmatize(tok)
        for tok in tokens if tok not in stop_words
    ]
    return " ".join(tokens)

df["clean_text"] = df["text"].apply(clean_text)
df = df[df["clean_text"].str.strip() != ""].reset_index(drop=True)
```

05 Feature Engineering with TF-IDF

TF-IDF converts cleaned text into a sparse numerical representation based on term importance. The supplied configuration limits the vocabulary to 20,000 features, combines unigrams and bigrams, ignores terms appearing in fewer than three documents, and excludes terms appearing in more than 90% of documents.

```
vectorizer = TfidfVectorizer(
    max_features=20000,
    ngram_range=(1, 2),
    min_df=3,
    max_df=0.9,
)

X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)

print(X_train_tfidf.shape)
print(X_test_tfidf.shape)
```

Result: 14,632 × 20,000 training matrix and 3,658 × 20,000 test matrix.

06 Model Development & Benchmarking

The notebook evaluates three classical supervised-learning baselines. Their reported test accuracy is presented without changing or reinterpreting the source results.

Model	Accuracy	Role in benchmark
Multinomial Naive Bayes	74.47%	Fast probabilistic baseline
Logistic Regression	75.94%	Linear classification baseline
Linear SVM	76.22%	Selected model in source run

```
models = {
    "Multinomial Naive Bayes": MultinomialNB(),
    "Logistic Regression":
        LogisticRegression(max_iter=1000, n_jobs=-1),
    "Linear SVM": LinearSVC(),
}

results = {}

for name, model in models.items():
    model.fit(X_train_tfidf, y_train)
    preds = model.predict(X_test_tfidf)
    acc = accuracy_score(y_test, preds)

    results[name] = {
        "model": model,
        "preds": preds,
        "accuracy": acc
    }

print(f"{name}: accuracy = {acc:.4f}")
```

The notebook automatically selects the model with the highest measured accuracy; in the supplied run this is Linear SVM with 0.7622 accuracy.

07 Evaluation

The selected model is evaluated using accuracy, precision, recall, F1-score and a confusion matrix. The source output reports approximately 0.76 overall accuracy, 0.75 macro-average F1 and 0.76 weighted-average F1.

Selected metric	Reported value
Accuracy	0.76
Macro-average precision	0.76
Macro-average recall	0.75
Macro-average F1	0.75
Weighted-average F1	0.76

08 Model Persistence & Reusable Inference

The model and TF-IDF vectorizer are persisted using Joblib. This separates model training from operational inference, so new text can be classified without repeating the complete training process.

```
joblib.dump(best_model, "best_text_classifier.joblib")
joblib.dump(vectorizer, "tfidf_vectorizer.joblib")

print("Saved: best_text_classifier.joblib")
print("Saved: tfidf_vectorizer.joblib")
```

Inference function

```
def predict_category(raw_text,
                    model=best_model,
                    vec=vectorizer):
    cleaned = clean_text(raw_text)
    vectorized = vec.transform([cleaned])
    prediction = model.predict(vectorized)[0]
    return prediction

prediction = predict_category(
    "The new graphics card improves rendering performance."
)

print(prediction)
```

09 NLP-Driven Automation Use Cases

Use case	Automation flow
Email classification	Incoming email → category → routing / folder
Support-ticket routing	Ticket text → issue type → assignment queue
Document tagging	Document → topic → metadata/tag
Feedback classification	Feedback → category → dashboard/workflow
FAQ routing	User query → topic → relevant knowledge area
Lead/message categorization	Message → category → downstream process

Scope note: the uploaded notebook demonstrates classification and inference. The business automation use cases above are logical extensions of that architecture and are not claimed as implemented or measured in the source.

10 Extension Roadmap

The source notebook identifies several directions for extending the baseline system. These are future implementation options rather than completed experiments in the supplied work.

Area	Proposed extension
Custom data	Replace the benchmark with a CSV containing text and label columns
Deep Learning	Embedding + LSTM/CNN using Keras/TensorFlow
Transformers	Fine-tune a pretrained model such as DistilBERT
Optimization	GridSearchCV / RandomizedSearchCV
Imbalanced data	Class weighting or resampling

11 Technical Conclusion

The supplied implementation demonstrates a clean separation of data preparation, NLP preprocessing, feature engineering, model training, evaluation, persistence and inference. The reported Linear SVM result provides a reproducible classical baseline that can serve as a starting point for domain-specific automation. Moving toward operational automation would require a task-specific labeled dataset, deployment interface, monitoring, error analysis and validation against the intended domain.